

HW2: Implementing Parallel Matrix Multiplication

1. 목표: 행렬 곱셈(GEMM)과 Freivalds Algorithm을 parallel로 구현하고 최적화를 한다.

2. Why GEMM & GEMV are important these days

GEMM(General Matrix Multiplication)과 GEMV(General Matrix-Vector Multiplication)는 Deep Learning, Computer Vision 등 AI 분야에서 학습을 위해 매우 많이 사용되는 연산이다.

Deep Learning 모델의 경우에는 gradient를 구하고, Error Rate를 최소화하기 위해 상당히 많은 iteration을 필요로 한다. 이때, gradient를 구하는 연산이 바로 Matrix Multiplication이다.

기본적인 Matrix Multiplication 알고리즘은 3중 for문을 사용하기 때문에 $O(N^3)$ 의 복잡도를 가진다. $O(N^3)$ 의 복잡도는 매우 compute intensive이므로 power 및 시간적으로 굉장히 많은 overhead를 가진다. 그럼에도 불구하고, AI모델을 training 시킬 때 필수적으로 사용되는 연산이기 때문에, 이 연산의 속도를 빠르게 하는 것이 매우 중요하다. 효율적인 병렬화와 CPU를 사용 시 cache에 기반한 최적화는 Matrix Multiplication 연산의 속도를 상당히 개선시킨다.

3. How did you implement your parallel algorithm

1. Parallel GEMM (General Matrix Multiplication)

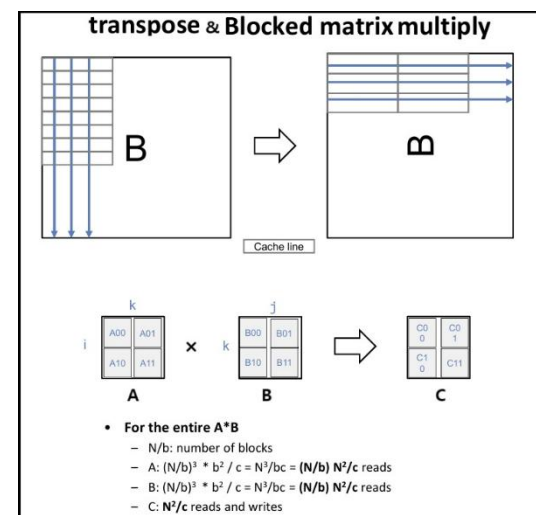
GEMM 연산의 속도를 높이기 위하여, multi-threading 과 cache utilization을 적용하고, 추가적으로 Loop Unrolling을 하였다.

[Using multi-threading]

먼저, multi-threading은 CPU의 spec에 맞추어 최종적으로 32개의 thread를 생성하였다.

실습에 이용하는 CPU는 2 Socket, 32 Physical Cores, 128 Threads를 지원하기에 최적의 threads가 될 수 있는 Thread 개수는 32, 64, 128이 된다. 이 중 어느 것을 선택했을 때 가장 성능이 좋은지는 Context Switching Overhead, CPU Scheduling Overhead 등을 고려하여야 하기 때문에, 직접 실험을 통해서 결정하였다. 더불어, 16 threads도 추가적으로 고려하였다. 왜 16개를 고려하였는지는 GEMV 구현을 설명 시 같이 설명하였다.

multi-threads를 이용하여 병렬화 한 부분은 3중 for문 중 가장 outer loop에 위치하는 i 를 병렬화 하였다. 즉, Matrix의 row들을 병렬화 하여, 32개의 thread를 사용한다고 가정 시 각 thread가 총 64개의 row를 연산 하게 하였다.



i를 선택한 이유는 마지막, C[i]를 계산 시 i에 의해서 dependency가 결정되기 때문이다. thread가 독립적으로 각각의 i를 맡아서 연산을 수행하면 thread간 dependency가 존재하지 않아, race condition이 발생하지 않고, 그에 따른 atomic 연산 overhead도 발생하지 않는다. 또한, i를 선택하면 single thread 기준으로 row wise 계산을 하기에 cache를 연속적으로 접근하여, spatial locality가 좋다. 이러한 이유들로 i를 선택하여 병렬화 하였다.

[Using transpose & Blocked Multiplication]

다음으로, GEMM 연산의 cache 효율성을 위하여 1page의 그림과 같이 transpose 및 Blocked Matrix Multiply를 적용하였다. 이를 통해 spatial locality를 향상시켰다.

Blocked Matrix Multiply에서 최적의 Block 크기를 결정하기 위하여 CPU cache의 spec을 확인한다.

\$L1d는 총 2MiB의 Memory Space를 가지고, 총 64개의 core가 존재하므로, 1개의 physical core 당 32KiB만큼의 Memory Space가 존재하는 것이다.

Caches (sum of all):	
L1d:	2 MiB (64 instances)
L1i:	2 MiB (64 instances)
L2:	32 MiB (64 instances)
L3:	256 MiB (16 instances)

또한, Matrix는 double형 data로 이루어져 있고, double형은 8Byte이므로, 32KiB / 8B = 4096B이다. 따라서, Block size는 (1thread 당 Block 개수) × (row 개수 × column 개수) = 4096를 가지도록 설정하면, 이론적으로 최적의 Block Size가 된다.

따라서, GEMM에서는 1개의 Block이 4096B 이하의 크기를 갖도록 하였다.

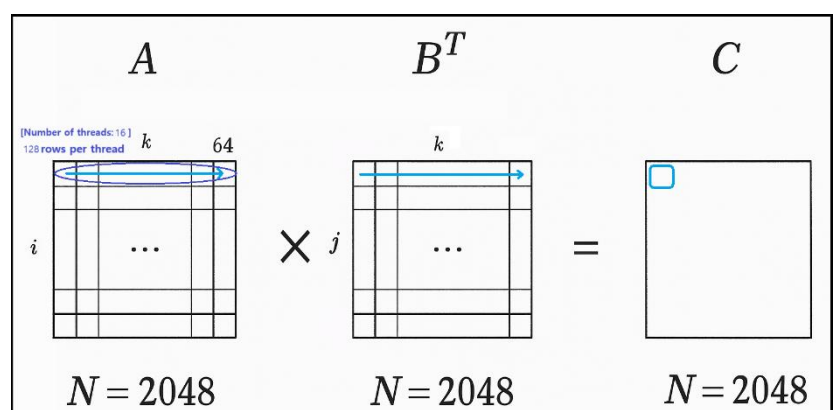
[Using Loop Unrolling]

추가로, 성능을 좀 더 개선하기 위하여 for문의 연산을 4개 단위로 Loop Unrolling을 하였다. 이를 통해 for문의 조건 검사, 즉, Branch Instruction을 감소시켰고 Instruction-Level Parallelism을 향상시켰다.

4개 단위로 Loop Unrolling을 한 이유는 실습에서 사용하는 "AMD EPYC 7452 32-Core Processor"는 AVX2의 256-bit SIMD를 지원한다. 256bit이므로 double형(64bit) 4개까지 ILP가 적용될 것이다. 이번 실습에서 compiler 옵션을 수정하지 못하기 때문에 AVX2를 직접적으로 사용하지는 못하지만, 기본 g++ compiler에서 자동으로, SIMD를 적용하지 않을까 싶어 Loop Unrolling을 4개 단위로 설정하였다.

본인이 구성한 GEMM optimization을 정리하면 아래 그림과 같다.

[threads 개수: 16일 때의 그림]



2. Parallel Freivalds' Algorithm

Freivalds' Algorithm을 이용한 행렬 연산 검사는 GEMV 연산을 수행한다. 이 GEMV 연산의 성능을 높이기 위하여, GEMM과 똑같이 multi-threading 과 cache utilization을 적용하고, 추가적으로 Loop Unrolling을 하였다.

먼저, Freivalds' Algorithm $[A \times (B \times v) = C \times v]$ 를 사용하기 위해서는 init_vector인 v 를 생성해야 한다. 참고 자료로 주어진 Wikipedia의 "Generate an $n \times 1$ random 0/1 vector r "에 따라, 0과 1의 원소를 가지는 $N \times 1$ Vector를 생성하였다. 더불어, main.cpp에서 init_vec() 함수의 실행시간은 성능평가에 들어가지 않기에 단순한 single thread for loop으로 vector를 생성하였다.

GEMV 성능향상을 위해서는 GEMM과 똑같은 방법을 사용하였다.

[Using multi-threading]

GEMM과 마찬가지로 실습에 이용하는 CPU는 2 Socket, 32 Physical Cores, 128 Threads를 지원하기에 최적의 threads가 될 수 있는 Thread 개수는 32, 64, 128이 된다. 이 중 어느 것을 선택했을 때 가장 성능이 좋은지는 Context Switching Overhead, CPU Scheduling Overhead 등을 고려하여야 하기 때문에, 직접 실험을 통해서 결정하였다

하지만, 실험 결과 16개의 thread와 32개의 thread의 성능이 비슷하거나 16개의 thread를 생성하였을 때가 성능이 더 좋게 나왔다.

이는 thread가 많아지면서 Context Switching Overhead, CPU Scheduling Overhead, false sharing의 발생 빈도가 높아졌을 수도 있기 때문이라고 추론된다. 더불어, "AMD EPYC 7452 32-Core Processor"는 각각 32개의 core를 가진 NUMA 노드가 2개 존재하는데, 32개의 thread가 전부 같은 NUMA 노드에 얹혀있지 않을 시 다른 NUMA 노드의 메모리를 참조해야 하기에 Latency가 증가한다. 따라서, NUMA 노드 Architecture로 인한 Latency도 기여할 것이라고 추측한다.

GEMV와 GEMM의 속도에 있어서 가장 큰 차이점은 연산량이다. GEMV의 경우 $O(n^2)$ 의 복잡도를 가지므로, 연산은 훨씬 적게 한다. 따라서, 속도는 Memory 접근량에 따라 달라진다. 즉, 16개의 thread로도 GEMV의 연산은 빠르게 처리가 되고, 오히려 32개의 thread를 사용시, thread간의 경쟁으로 인하여 memory 접근이 증가하기에 속도가 저하되는 것이라고 추론된다.

그 결과, multi-threading은 CPU의 spec에 맞추어 16개의 thread를 생성하였다.

[Using transpose & Blocked Multiplication]

GEMM과 마찬가지로 $L1d$ 는 총 2MiB의 Memory Space를 가지고, 총 64개의 core가 존재하므로, 1개의 physical core 당 32KiB만큼의 Memory Space가 존재한다.

또한, Matrix 및 Vector는 double형 data로 이루어져 있고, double형은 8Byte이므로, $32\text{KiB} / 8\text{B} = 4096\text{B}$ 이다. 따라서, 1개의 Block이 row 개수 $\times$ column 개수 = 4096를 가지도록 설정하면, 이론적

으로 최적의 Block Size가 된다.

[Using Loop Unrolling]

추가로, GEMM에서와 같은 이유로 성능을 좀 더 개선하기 위하여 for문의 연산을 4개 단위로 Loop Unrolling을 하였다.

4. Which approach is better

이번 과제에 GEMM Approach와 Freivalds' Algorithm Approach를 비교할 때, Matrix Multiply 검증 측면에서는 Freivalds' Algorithm이 더 우수하다고 판단된다. 이는 다음과 같은 이유 때문이다.

[연산량 감소 및 빠른 속도]

Freivalds' Algorithm은 확률적인 방법을 사용하여 행렬 곱셈의 검증을 수행하기 때문에 완전한 행렬 곱셈(GEMM)에 비해 훨씬 적은 계산량으로 결과의 정확성을 높은 확률로 확인할 수 있다. 계산량은 다음과 같다.

$B \times v$	$O(n^2)$
$A \times (B \times v)$	$O(n^2)$
$C \times v$	$O(n^2)$

따라서, GEMM의 시간 복잡도는 $O(n^3)$ 인 반면, Freivalds' Algorithm은 $O(n^2)$ 이다.

[메모리 row wise 접근, spatial locality 최적화]

Freivalds' 알고리즘에서는 벡터 연산 위주로 구성되어 메모리 접근 패턴이 더 단순하고 일관되며, 캐시 활용성이 우수하여 GEMM 대비 성능 이점이 있다.

반면, Matrix Multiply 검증이 아닌 정확한 행렬 곱셈 결과가 필요한 상황이라면 GEMM을 사용해야 한다. 왜냐하면, Freivalds' Algorithm은 $A \times B$ 의 연산 결과인 C Matrix가 직접적으로 도출되지 않고, 검증 연산은 random vector를 사용하여 연산되기 때문이다. 즉, Freivalds' 알고리즘은 검증 용도로만 사용하는 것이 적합하다.

5. Reason Why Freivalds' algorithm is probabilistic and its error bound

Freivalds' 알고리즘은 세 개의 $n \times n$ 행렬 A, B, C 에 대해 $AB = C$ 인지 확률적으로 확인하는 알고리즘이다. 다음과 같은 절차로 이루어진다:

1. 무작위로 벡터 r 를 생성 (각 성분은 독립적으로 1/2 확률로 0 또는 1을 가짐)
2. $P = A \times (B \times r) - C \times r$ 를 계산
3. 만약 P 가 영벡터(모든 성분이 0)라면, $AB=C$ 일 가능성이 높으며, 그렇지 않다면 $AB \neq C$ 이다.

이 알고리즘이 확률적인 이유는 다음과 같다:

- 만약 $AB=C$ 라면, 위의 계산은 항상 영벡터를 결과로 주므로 항상 참이다.
- 그러나 $AB \neq C$ 일 경우, 오류를 감지하지 못할 확률은 최대 1/2이며, k 번 반복하면 오류 확률은 $(\frac{1}{2})^k$ 로 지수적으로 감소한다.

결국, Freivalds' 알고리즘의 오류 확률(error bound)은 $(\frac{1}{2})^k$ 이며, 원하는 신뢰성을 위해 반복 횟수를 조정할 수 있다. 이로 인해 계산 비용을 크게 줄이면서 높은 신뢰도를 확보할 수 있는 효율적인 방법이다.

[수학적인 증명은 다음과 같다]

Freivalds 알고리즘의 오차 확률을 p 라고 하자.

다음 두 가지를 case를 따져본다.

$A \times B = C$ 일 경우, 오류 확률 $p = 0$

$A \times B \neq C$ 일 경우, 오류 확률 $p \leq 1/2$

1) $A \times B = C$ 일 때

$$\vec{P} = A \cdot (B\vec{r}) - C\vec{r} = (A \cdot B)\vec{r} - C\vec{r} = (A \cdot B - C)\vec{r} = \vec{0}$$

$A \times B = C$ 이므로, $A \times B - C = 0$ 이 된다.

따라서 어떤 r 벡터를 선택하더라도 결과는 항상 영 벡터가 되며, 오류가 발생하지 않는다.

$$\Pr[\vec{P} \neq 0] = 0$$

2) $A \times B \neq C$ 일 때

$D = A \times B - C$ 라고 하자. D 는 0이 아닌 항을 하나 이상 포함한 행렬이다.

그러면 다음과 같이 표현 가능하다:

$$\vec{p} = D \cdot \vec{r} = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_n \end{bmatrix}$$

각 p_i 는 다음과 같이 계산된다:

$$p_i = \sum_{k=1}^n d_{ik} r_k = d_{i1}r_1 + \cdots + d_{ij}r_j + \cdots + d_{in}r_n = d_{ij}r_j + y$$

여기서 y 는 나머지 항들의 합이며, 상수로 간주할 수 있다.

$$\Pr[p_i = 0] = \Pr[p_i = 0 \mid y = 0] \cdot \Pr[y = 0] + \Pr[p_i = 0 \mid y \neq 0] \cdot \Pr[y \neq 0] \quad (\text{Bayes 정리})$$

만약 $y = 0$ 이면, $p_i = d_{ij}r_j$ 이므로:

$$\Pr[p_i = 0 \mid y = 0] = \Pr[r_j = 0] = \frac{1}{2}$$

만약 $y \neq 0$ 이라면, $p_i = d_{ij}r_j + y$ 이 0 이 되기 위한 조건은 매우 제한적이다:

$$\Pr[p_i = 0 \mid y \neq 0] = \Pr[r_j = 1 \wedge d_{ij} = -y] \leq \Pr[r_j = 1] = \frac{1}{2}$$

위 식을 대입하면,

$$\Pr[p_i = 0] \leq \frac{1}{2} \cdot \Pr[y = 0] + \frac{1}{2} \cdot (1 - \Pr[y = 0]) = \frac{1}{2}$$

$$\Pr[\vec{p} = \vec{0}] = \Pr[p_1 = 0 \wedge p_2 = 0 \wedge \cdots \wedge p_n = 0] \leq \Pr[p_i = 0] \leq \frac{1}{2}$$

즉, $A \times B \neq C$ 인 경우에도 랜덤 벡터 r 하나를 기준으로 전체 벡터 $\vec{p} = \vec{0}$ 이 될 확률은 최대 $1/2$ 이다.

6. Reasons why your program is performing at its best and proving whether performance margin is there or not

이번 과제에서는 multi-threading, cache utilization, Loop Unrolling을 성능 개선의 주요한 방법으로 사용하였다. 앞서 3번 문항에서 어떻게 Program을 구성하였는 지 및 구성한 이유에 대해 설명하였지만, 개선 사항이 존재하였기에 추가로 설명한다.

6-1. Explaining How did I implemented and improved my code

[Algorithm]

GEMM의 경우에는 앞서 **3. How did you implement your parallel algorithm**에서 구현한 방법으로 구현 시 과제의 달성 조건이었던 10초 이내 및 4.2초 이내를 달성하였다.

하지만, GEMV의 경우 **3. How did you implement your parallel algorithm**에서 구현한 방법으로 구현 시 과제의 달성 조건인 0.05초는 달성할 수 있었지만, 0.0015초는 달성하기 어려웠다. 따라서, 다른 Algorithm을 시도하였다.

1) 기존 방법: Using row wise thread

기존에 구현한 방법은 multi-thread로 i, j 중 i 를 병렬화 하는 방법이었다. 이 방법의 장점은 A 의 row를 연속적으로 접근하고, B 의 row도 연속적으로 접근하여 Cache Spatial locality를 잘 활용할 수 있다는 점이였다. 더불어, 모든 threads들이 i 에 대해 independent하므로, race condition을 고려할 필요가 없었다.

하지만, 단점은 각각의 thread가 같은 B 를 참조하는데, B 의 메모리 영역을 참조하기에 각각의 thread가 각각 가지고 있는 $L1$ 이 아니라 다른 thread가 가지고 있는 $L1$ 을 참조하여 이에 따른 심각한 latency가 발생할 것으로 예상된다.

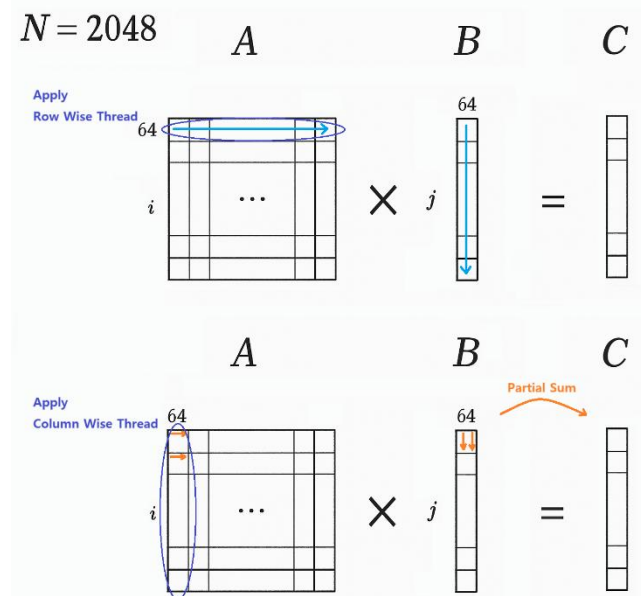
2) Using row wise thread but changing the loop order

더불어, Blocked Matrix Multiply에서 Loop 순서를 바꿔보았다. Row wise thread 및 Blocked Matrix Multiply를 사용 시 loop의 순서를 $ii \rightarrow jj \rightarrow i \rightarrow j$ 로 구성을 하는 것과 $ii \rightarrow i \rightarrow jj \rightarrow j$ 로 구성하는 것의 성능의 차이가 거의 존재하지 않았다.

3) Using column wise thread

위의 그림의 아래쪽 Matrix Vector Multiplication이 Column Wise threading을 이용한 방법이다. 2048의 Column을 Threads의 수로 나누고, single thread 기준 할당된 열을 다시 row wise로 처리하는 방법이다.

이 방법의 장점은 각각의 thread가 B 를 각각 참조하므로, thread끼리 같은 B 의 element를 공유는



문제가 없어진다.

하지만, Column wise로 thread가 연산을 진행하기 때문에, 최종 $c[i]$ 를 도출하기 위해 thread에서 partially 연산된 결과를 나중에 합쳐줘야 Race Condition을 피할 수 있고, 이에 따른 overhead가 발생한다.

4) Using row wise thread(기존 방법) + copying partial B Vector to each thread

column wise thread를 사용하는 것은 많은 단점을 동반한다. 따라서, 각각의 서로 다른 thread가 같은 B를 참조하는 문제를 해결하기 위해, single thread 기준으로 연산을 시작할 때, 연산할 B Vector의 부분을 자기 자신이 속해 있는 core의 $L1$ 으로 복사해오는 것이다. 그 결과 single thread를 시작할 때, Block Size만큼의 작은 배열을 만들어 복사해오는 overhead가 존재하지만, 그 이후부터는 각각의 thread가 복사해온 자신의 local B vector를 사용하기 때문에 $L1$ cache를 충분히 활용 가능하고, Hit Rate도 상당히 올라갈 것으로 예상된다.

GEMV에서는 1), 2), 3), 4)번 중 이론적으로 가장 cache를 잘 활용할 수 있는 4)번을 최종적으로 선택하였다.

더불어 Using row wise thread(기존 방법) + copying partial B to each thread는 GEMM에서도 사용 가능하다. 물론 GEMM에서는 Matrix를 A, B 모두 복사해야 $L1$ 을 utilize할 수 있기에, 복사해오는 overhead가 클 것이다. 이는 실험적으로 GEMM에서 "기존 방법"이 나은지, "기존 방법 + copying"이 나은지 결정한다.

실험결과 GEMM에서는 기존 방법 + copying이 근소한 성능 향상을 보였기에, 기존 방법 + copying을 최종 알고리즘으로 선택하였다.

[multi-threading]

실습에 이용하는 CPU는 2 Socket, 32 Physical Cores, 128 Threads를 지원하기에 최적의 threads가 될 수 있는 Thread 개수는 32, 64, 128이 된다. 이 중 어느 것을 선택했을 때 가장 성능이 좋은지는 Context Switching Overhead, CPU Scheduling Overhead, false sharing 등을 고려하여야 하기 때문에, 직접 실험을 통해서 결정하였다

또한 생각해야 할 것은 연산량이다. 만약 내가 실행하고자 하는 Matrix Multiplication의 크기가 작다면, 연산량이 적을 것이고 그 경우 Context Switching Overhead, CPU Scheduling Overhead, false sharing등이 발생하는 빈도가 적어져, 32개의 thread보다 16개의 thread가 더 좋은 성능을 보였다.

최종적으로 GEMM에서는 32 threads, GEMV에는 16 threads를 선택하였다.

[Cache Spatial Locality 최적화]

Matrix 연산 시 Blocked Matrix Multiplication을 통해 $L1$ 에서 감당가능한 Element 별로 연산하여 $L1$ cache Hit Rate를 향상시켰다. 또한, Transpose Matrix를 이용하여 B Matrix에서도 row wise 접근

을 할 수 있게 하여 spatial locality를 극대화하였다.

Block size는 32개의 threads를 사용 시 (1thread 당 Block 개수) $\times$ (row 개수 $\times$ column 개수) = 4096가 되도록 설정한다고 하였다.

만약 16개의 threads를 사용한다면, (1thread 당 Block 개수) $\times$ (row 개수 $\times$ column 개수) = 8192 이면 될 것이다.

GEMM의 Block Size는 copying을 고려하여 \$L1에 local_a, local_b, local_c가 모두 들어가도록 설정한다. 즉, (1thread 당 Block 개수) $\times$ (row 개수 $\times$ column 개수) = 3 $\times$ (row 개수 $\times$ column 개수) 이므로, (row 개수 $\times$ column 개수) $\leq$ 1300 정도로 설정하면 된다. 즉, 32 $\times$ 32 또는 16 $\times$ 16 정도로 설정하는 것이 좋을 것이다. 최종적으로 Block Size 32 $\times$ 32 과 16 $\times$ 16 중 실험적으로 더 좋은 성능을 나타낸 "16 $\times$ 16"을 Block Size로 결정하였다.

GEMV의 Block Size는 copying을 고려하여 \$L1에 local_b가 들어가면 된다. 따라서, (1thread 당 Block 개수) $\times$ (row 개수 $\times$ column 개수) = 1 $\times$ (row 개수 $\times$ column 개수) 이므로, (row 개수 $\times$ column 개수) $\leq$ 8192 정도로 설정하면 된다. 따라서, 8192보다 절반의 공간만 활용하지만, 실제 실험에서 좋은 성능을 보였던 "Block Size 64 $\times$ 64"을 최종적으로 선택하였다.

[Loop Unrolling]

실습에서 사용하는 "AMD EPYC 7452 32-Core Processor"는 AVX2의 256-bit SIMD를 지원한다. 256bit이므로 double형(64bit) 4개까지 ILP가 적용될 것을 예상하여, 4개씩 Loop Unrolling 적용하였다.

6-2. Theoretical performance improvement calculation

Memory로부터 몇 번의 read 와 write를 할 지 계산한다. Cache Hit은 0번 read, write로 가정한다.

1) GEMM

Naïve GEMM Performance

- Number of read and write operations based on a single thread (N=2048)
 - A: N^2/c reads
 - B: N^3 reads (Since Not using Transpose, Assume Always Cache evicted when accessed)
 - C: N^2/c reads + N^2/c writes
- Using Single Threads
- Not Using Block multiply

Calculated My GEMM Performance

- Number of read and write operations based on a single thread
 - A: $(N/b)^3 * b^2 / c = N^3/bc = (N/b) N^2/c$ reads
 - B: $(N/b)^3 * b^2 / c = N^3/bc = (N/b) N^2/c$ reads
 - C: N^2/c reads and writes
- Using 32 threads (read or writes consume x32 less time compared with Using only 1 thread)
- Block Size: 16

2) GEMV

Naïve GEMV Performance

- Number of read and write operations based on a single thread (N=2048)
 - A: N^2/c reads
 - B: N^2 reads
 - C: N / c writes
- Using Single Threads
- Not Using Block multiply

My GEMV Performance:

- Number of read and write operations based on a single thread (N=2048)
 - A: $(N/b)^2 * b^2/c = N^2 / c$ reads
 - B: $(N/b) * b/c = N / c$ reads
 - C: N / c writes
- Using 16 Threads (read or writes consume x16 less time compared with Using only 1 thread)
- Block Size: 64

7. Evaluation including tables or graphs

7-1. GEMM Evaluation

Threads 및 Block Size를 결정하기 위한 Evaluation

Block Size 16 * 16	실행 횟수	16 Threads	32 Threads	64 Threads	128 Threads
Execution time (sec)	1	1.79606	1.08622	1.23595	1.61531
	2	1.72995	0.939127	1.20239	1.43105
	3	1.74806	1.16181	1.14496	1.4464
	4	1.68866	0.98621	1.19532	1.4528
	5	1.69763	1.0382	1.31324	1.48918
AVG		1.73E+00	1.04E+00	1.22E+00	1.49E+00

Block Size 32 * 32	실행 횟수	16 Threads	32 Threads	64 Threads	128 Threads
Execution time (sec)	1	1.74539	1.00441	1.11277	실행불가
	2	1.93669	0.988784	2.13011	실행불가
	3	1.80457	1.11775	1.07767	실행불가
	4	1.88547	1.08644	1.1212	실행불가
	5	1.76621	1.0857	1.11396	실행불가
AVG		1.83E+00	1.06E+00	1.31E+00	

Block Size 64 * 64	실행 횟수	16 Threads	32 Threads	64 Threads	128 Threads
Execution time (sec)	1	1.89657	1.05427	실행불가	실행불가
	2	1.88709	1.096	실행불가	실행불가
	3	1.89339	1.01159	실행불가	실행불가
	4	1.86909	1.03248	실행불가	실행불가
	5	1.88341	1.00551	실행불가	실행불가
AVG		1.89E+00	1.05E+00		

위 실험결과에 따라, 최종적으로 Block Size 16 * 16 및 32 threads로 결정하였다.

7-2. GEMV Evaluation

Threads 및 Block Size를 결정하기 위한 Evaluation

Block Size 64 * 64	실행 횟수	16 Threads	32 Threads	64 Threads	128 Threads
Execution time (sec)	1	0.0069	0.0071	0.0106	0.0188
	2	0.0067	0.0068	0.0104	0.0178
	3	0.0073	0.0067	0.0104	0.0192
	4	0.0070	0.0073	0.0110	0.0196
	5	0.0062	0.0082	0.0102	0.0190
AVG		0.00682	0.00722	0.01052	0.01888

Block Size 32 * 32	실행 횟수	16 Threads	32 Threads	64 Threads	128 Threads
Execution time (sec)	1	0.0073	0.0077	0.0107	0.0181
	2	0.0084	0.0078	0.0102	0.01886
	3	0.0082	0.0084	0.0108	0.01886
	4	0.0075	0.0080	0.0110	0.0165
	5	0.0076	0.0078	0.0110	0.0179
AVG		0.0078	0.00794	0.01074	0.018044

Block Size 16 * 16	실행 횟수	16 Threads	32 Threads	64 Threads	128 Threads
Execution time (sec)	1	0.0075	0.0071	0.0106	0.0102
	2	0.0070	0.0069	0.0115	0.0106
	3	0.0071	0.0070	0.0104	0.0104
	4	0.0070	0.0073	0.0102	0.0106
	5	0.0071	0.0080	0.0103	0.0099
AVG		0.00714	0.00726	0.0106	0.01034

위 실험결과에 따라, 최종적으로 Block Size 64 * 64 및 16 threads로 결정하였다.

8. Something else

8-1. GEMV Advanced Performance 달성에 대한 방법

이번 과제에서 GEMV Advanced Performance의 제한이었던 0.0015초는 달성하지 못하였다. 먼저, 이번 과제에서 고려한 요소는 다음과 같다.

GEMM

- 1) Cache의 spatial locality를 고려하여, B를 transpose

GEMM & GEMV

- 1) Race Condition, Thread scheduling, Context Switching을 모두 고려한 Multi-Thread 개수
- 2) L1 Cache를 최대한 활용하기 위한 Blocked Matrix Multiplication
- 3) False Sharing을 방지하기 위한, local Matrix 또는 local Vector를 만들어 연산
- 4) Loop Unrolling을 통한 SIMD 유도

CPU관점에서는 연산 뿐만 아니라, cache utilization이 성능에 매우 큰 영향을 미치기에, cache를 최적화하기 위한 code 구성을 많이 고려하였다.

이번 과제에서 연산의 속도를 가속시키는 것은 multi-threading을 이용하여 달성하였고, cache 최적화는 Blocked Matrix 및 Transpose 및 copying Matrix등을 이용하여 달성하였다.

하지만, 현재 본인의 code에서는 thread 관리를 제대로 못하였는지, 실험을 통해 구한 최적의 thread개수가 GEMM에서는 32개, GEMV에서는 16개로 도출되었다.

CPU의 spec인 2 Socket, 32 Physical Cores, 128 Threads에 비하면, 한참 못 미치는 thread 개수인 것이다.

따라서, GEMV Advanced Performance를 추가적으로 달성을 위해서는 thread관리가 필수적이라고 생각된다. 현재 GEMV의 속도는 0.0068초가량 나왔는데, Advanced Performance인 0.0015초를 위해서는 약 4~5배가 빨라져야 하고 이는 16개의 thread를 64개 또는 128개로 늘렸을 때의 이론적인 성능 향상 이론 값과 비슷하다. (64 threads 시 16 threads보다 x4 빠름, 128 threads 시 x8빠름)

즉, thread overhead에 따른 code 구성을 하였다면, Advanced Performance를 달성하였을 것으로 생각된다.

9. Reference

- [난수 선택]: <https://modoocode.com/304>

- 2_thread_mgp_2025.pdf

- 3_mat_mul_mgp_2025.pdf